Microsoft Sentinel and Defender for Endpoint Implementation Manual

A practical build sequence for endpoint onboarding, Sysmon, SIEM rules, Fusion, hunting, and automated response. October 2026.

1. Decide the architecture before clicking Connect

Use one pilot Microsoft Entra tenant, Azure subscription, resource group, Log Analytics workspace, and primary Microsoft Sentinel workspace. Connect that workspace to the Microsoft Defender portal. The Defender portal supplies the unified incident queue; Defender XDR correlates Defender alerts; the primary Sentinel workspace can contribute Sentinel alerts to cross-source correlation. Secondary Sentinel workspaces do not provide the same cross-source correlation with Defender alerts. Record licensing, data residency, retention, daily ingestion cost, ownership, and approved response authorities.

Data path A: endpoint -> Defender for Endpoint sensor -> Defender XDR alerts, incidents, and advanced hunting. Data path B: Windows Security Events and Sysmon -> Azure Monitor Agent (AMA) and Data Collection Rules (DCR) -> Log Analytics tables -> Sentinel KQL. Data path C: network device or Suricata -> Syslog/CEF forwarder or supported custom ingestion -> Sentinel tables. The incident workflow links these sources by device, user, IP, process, and time. A Logic App playbook adds context and routes a decision; Defender provides approved endpoint actions.

Create an inventory of pilot devices and expected telemetry before rollout. Include device hostname, Defender device ID, Azure/Arc resource ID, operating system, owner, criticality, network zone, clock source, expected tables, and approval contact. A device can have a healthy Defender sensor without AMA or Sysmon; these are separate onboarding paths.

2. Permissions, licensing, and cost controls

The workspace creator needs suitable Azure permissions. Sentinel analysts, content managers, and playbook operators need their respective roles at the correct resource-group or resource scope. A Logic App is a separate Azure resource, with its own connection and identity permissions. Give the incident playbook only the rights required for the operations it performs. Define who can approve endpoint isolation, user disablement, firewall blocks, email purge, and actions affecting critical or OT assets.

Estimate daily gigabytes per source, especially Sysmon, Defender advanced hunting streams, firewall, and Suricata. Enable Azure budget alerts and inspect Log Analytics ingestion after the first day. Start with measured DCR event selections rather than all events everywhere. Decide whether Defender raw advanced hunting tables must be ingested into Sentinel; unified Advanced hunting may already provide useful investigation access, while workspace ingestion can add cost.

Acceptance record: subscription, region, workspace, primary/secondary status, roles, cost owner, baseline volume, retention, pilot devices, and rollback contacts.

3. Create the Log Analytics workspace and Sentinel

In Azure, create the Log Analytics workspace in the planned region and resource group. Review workspace access, retention, and daily cap policy. Add Microsoft Sentinel to that workspace. In the Defender portal, go to Settings > Microsoft Sentinel > Workspaces or Connect a workspace as shown for your tenant. Connect the pilot workspace and select it as primary. New customers may be connected automatically. Confirm that Analytics, Incidents, Hunting, Content hub, Automation, and Logs are accessible to the intended team.

Install Content hub solutions for Windows Security Events, Syslog/CEF, and the other pilot sources. Read each solution's connector page because prerequisites and table destinations differ. Do not enable every template before a source exists. Record connector owner and a test query for each source.

Verify: a permitted analyst can open the intended workspace; another analyst with responder rights can access an incident; the playbook resource group is visible to its owner.

4. Onboard Windows clients and servers to Defender for Endpoint

Use the Defender portal onboarding page to choose a supported method for each device class. For Intune-managed Windows clients, establish the Defender service connection, create an Endpoint detection and response policy, target a small device group, and check its Device status. Use the appropriate supported deployment path for Windows servers. A local script is useful for a small controlled test but is not the default enterprise deployment method.

In Assets > Devices, verify the pilot device appears. Open its device page and confirm onboarding status, sensor health, last seen time, device timeline, OS, risk, and exposure context. Run a Microsoft-approved detection test or a SOC-approved controlled simulation. Confirm the event on the device timeline and any expected alert or incident. Check actual Defender telemetry with a small Advanced hunting query:

```
DeviceProcessEvents
| where Timestamp > ago(24h)
| summarize Events=count(), LastSeen=max(Timestamp) by DeviceName
| order by LastSeen desc
```

If the table is empty, verify that Defender for Endpoint is deployed and that the chosen portal and permissions expose the table. Do not infer device protection or reporting from an Intune policy success message alone. Record sensor health and event visibility per device.

5. Connect Defender XDR with Sentinel

When Sentinel is connected to the Defender portal and Defender XDR is licensed, review the automatically configured Defender XDR integration. For a Sentinel workflow in the Azure portal, enable the Microsoft Defender XDR data connector and choose alert/incident synchronization and any raw advanced hunting tables required. Review duplicate alert connectors: the XDR integration changes how integrated Defender product alerts are brought in, and separate Microsoft incident-creation rules may be disabled to prevent duplicate incidents.

Open a Defender for Endpoint alert and its Defender XDR incident. In Sentinel or the unified Defender incident queue, confirm the corresponding incident, alert source, device/user entities, timestamps, status, and link to the Defender device. Changes to supported incident fields can synchronize between services. Run a controlled Sentinel analytic alert on the same pilot entity and inspect whether the primary-workspace alert is associated in the unified experience. Correlation depends on alert relationships and engine behavior; do not promise merging from a single matching hostname.

Choose raw event tables deliberately. DeviceProcessEvents, DeviceNetworkEvents, DeviceFileEvents, and DeviceInfo can support richer KQL, but ingesting advanced hunting events into Log Analytics is distinct from using them in Defender Advanced hunting and may be billable. Some Defender TVM tables are not streamed into Sentinel even if they appear in a schema browser. Verify table availability by querying it.

6. Onboard Windows Security Events with AMA

For Azure VMs or Azure Arc-enabled servers, use Windows Security Events via AMA. Create a DCR in the connector, select the correct workspace destination, choose Common/Minimal or a custom event set, and associate the approved pilot machines. Validate AMA extension status and DCR association on each device. The Windows Security Events connector writes to SecurityEvent, which many Sentinel templates query.

```
SecurityEvent
| where TimeGenerated > ago(24h)
| summarize Count=count(), Last=max(TimeGenerated) by Computer, EventID
| order by Last desc
```

Test expected events such as a controlled failed sign-in. Compare the local event time to TimeGenerated and inspect Account, Computer, IpAddress, and other fields. No event may mean the source did not generate it, the DCR did not select it, AMA is unhealthy, or ingestion is delayed. Narrow the failure by checking each layer.

7. Deploy customized Sysmon and collect it correctly

Define which behaviors Sysmon must make visible. A starter configuration can include process creation (1), network connection (3), file creation (11), registry changes (12-14), and DNS query (22), subject to supported Sysmon version and your noise budget. Tune inclusion/exclusion rules by device role and test locally. Keep the configuration in version control with owner, change note, rollback, and expected event-rate impact. Do not enable every event type across an estate merely because storage is available.

Install Sysmon on an approved pilot endpoint with the controlled configuration. Verify local events under Microsoft-Windows-Sysmon/Operational. Confirm parent/child process, command line, ProcessGuid, network destination, DNS query, file and registry details as appropriate. Then collect that event channel through an AMA Windows events DCR on an Azure/Arc eligible host. Use a tested XPath selector for the chosen event IDs. Check the DCR's destination: generic Windows event collection commonly appears in WindowsEvent, not SecurityEvent.

```
WindowsEvent
| where TimeGenerated > ago(24h)
| where Provider == "Microsoft-Windows-Sysmon"
| summarize Count=count(), Last=max(TimeGenerated) by Computer, EventID
| order by Last desc
```

Inspect the actual Provider field, EventData, and RawEventData in your workspace. Validate the XPath locally with Get-WinEvent before adding it to a DCR. Avoid joining Defender and Sysmon by PID alone because PIDs are reused; add hostname, process creation time, process path, user, and where available stable process identifiers. Sysmon is extra telemetry for investigation, while Defender for Endpoint remains the endpoint detection and response service.

8. Add network, identity, DNS, and email sources

Install Syslog via AMA or CEF via AMA from Content hub. Configure a Linux source or log forwarder, DCR, facility/severity selections, and device forwarding. Plain Syslog generally arrives in Syslog, CEF in CommonSecurityLog. Compare raw source logs to parsed action, direction, source/destination IP and port, device name, signature, and timestamp. A direction or action parser defect can turn denied inbound traffic into a false outbound command-and-control story.

Suricata EVE JSON requires an approved custom-log ingestion path or a tested transform to Syslog/CEF. It does not automatically populate CommonSecurityLog. Record the parser, table, fields, and event examples. Add relevant Entra sign-ins, Microsoft 365/Defender email alerts, DNS, and firewall sources. Normalize asset, user, IP, and time conventions. Verify events from every selected source before building joins or automation.

```
CommonSecurityLog
| where TimeGenerated > ago(24h)
| summarize Count=count(), Last=max(TimeGenerated) by DeviceVendor, DeviceProduct
```

9. Create rule 1: repeated failed Windows sign-ins

Open Analytics and review a Content hub scheduled rule template first. If building a pilot rule from scratch, verify the SecurityEvent schema and run this query manually:

```
SecurityEvent
| where TimeGenerated > ago(1h) and EventID == 4625
| summarize Failures=count(), First=min(TimeGenerated), Last=max(TimeGenerated)
    by Account, Computer, IpAddress
| where Failures >= 10
```

Create a Scheduled query rule. Name the hypothesis, select severity, map relevant MITRE ATT&CK behavior, run every 15 minutes with a one-hour lookback, set the alert threshold, map Account/Host/IP entities, and choose alert grouping and incident creation. Because lookback windows overlap, test for duplicate alerts. Generate an authorized failure pattern on a pilot account and inspect the raw evidence and incident. Validate expected benign failures; tune thresholds or exceptions narrowly. This rule indicates an investigation lead, not confirmed credential compromise.

10. Create rule 2: suspicious PowerShell using Defender endpoint events

In Defender Advanced hunting or a supported Sentinel query surface where DeviceProcessEvents is available, test a behavior query before enabling a detection. Keep the first rule narrow and use command-line indicators as a lead rather than a verdict:

```
DeviceProcessEvents
| where Timestamp > ago(1h)
| where FileName in~ ("powershell.exe", "pwsh.exe")
| where ProcessCommandLine has_any ("-enc", "EncodedCommand", "DownloadString")
| project Timestamp, DeviceName, AccountName, FileName,
          ProcessCommandLine, InitiatingProcessFileName, SHA1
```

Pivot to DeviceNetworkEvents, device timeline, DNS, and Sysmon process/network records. Define expected administrative tools and false-positive conditions. Select a custom detection in the unified Defender experience when it better fits endpoint events; use a Sentinel scheduled rule when the required data and response path are in the workspace. Map device and account entities. Test with a controlled benign command and an approved simulation. Record query owner, source coverage, severity, response guidance, validation evidence, and rollback.

11. Rule engineering workbook and playbook contract

Every detection needs: hypothesis; data source/table; required fields; KQL; schedule/lookback; threshold; entity mapping; MITRE mapping; expected true-positive behavior; common false positives; test events; alert grouping; triage steps; escalation criteria; owner; version; review date; and rollback. Keep the detection source and the response playbook separate so analysts can change investigation steps without silently changing what generates alerts.

A response playbook needs a clear trigger and input contract: incident ID, workspace, alert source, host, user, IP, URL/hash when present, severity, confidence, and evidence links. Treat missing entities as a normal branch. Define enrichment outputs, human approval points, actions, retry/timeouts, error route, audit record, and rollback owner. Use minimal permissions. Document which connector supplies each field.

12. Configure Fusion after source signals are healthy

Fusion is the built-in Advanced Multistage Attack Detection rule, a machine-learning correlation engine for combinations of eligible alerts/activities. In Analytics > Active rules, filter Fusion and verify it is enabled. Review its source signals and severity filters. Keep relevant sources enabled unless a measured reason justifies narrowing. The newer Fusion version is marked preview in current Microsoft documentation.

Fusion is not a custom KQL rule. It will not necessarily fire from one test alert or merely because a device was onboarded. Check that relevant Defender and Sentinel alerts exist, source signals are enabled, and incidents have correct entities. When Fusion produces an incident, inspect constituent alerts, timeline, entities, and root evidence. Compare Fusion findings against your scheduled-rule coverage and decide whether a specific behavior needs its own rule.

13. Build the first incident-assist playbook in Logic Apps

Create a Logic App playbook with a Microsoft Sentinel incident trigger. Choose the correct resource group and approved authentication, preferably a managed identity where supported. Grant that identity only the permissions it needs. Build these actions in order: read incident details; extract alerts/entities; retrieve asset owner and criticality; add Defender device link; retrieve related identity/network/Sysmon context through approved queries; create analyst tasks; add a concise evidence comment; notify the team with an incident link.

Create an automation rule on incident creation. Scope it to a named pilot rule or a clear incident condition. Set order and expiration/review. Add Run playbook and grant the Sentinel service permission to run that Logic App. Test with a controlled incident. Inspect run history, each action's inputs/outputs, incident comment/task, and notification. Add failure branches for empty host, missing user, denied query, stale data, timeout, and duplicate runs.

Do not start with automatic containment. After the analyst-assist workflow is reliable, add an approval action for endpoint isolation, account disablement, firewall blocking, or broad message purge. The approval card should show device/user, evidence, business owner, predicted impact, and rollback path. Log approver and action result. Critical and OT assets require operational-owner coordination.

14. Incident scenario: endpoint PowerShell plus network signal

Defender for Endpoint identifies a suspicious process tree, command line, signer, and network connection. Defender XDR groups related endpoint/identity alerts. Sentinel contributes Windows Security Events, customized Sysmon process/DNS/network records, IPS or firewall action, and cloud identity events. A Sentinel analytic rule may add a narrow alert, and Fusion may later recognize a multistage pattern.

The incident playbook collects Defender device URL, process and user, Sentinel log pivots, Suricata signature/action, destination reputation, asset owner, and recent identity anomalies. It creates tasks to confirm execution, persistence, network direction, and scope. The analyst decides whether containment is justified. Approved response uses the Defender endpoint action through an authorized workflow. The case records decision, action, evidence, and follow-up detection tuning.

15. Incident scenario: phishing to endpoint follow-through

Begin with sender, recipient, message headers, URLs, attachment, delivery action, and user interaction. Correlate identity sign-ins and MFA events with Defender endpoint process/file/network activity, DNS, Sysmon, and firewall/IPS events. The playbook adds affected recipients and devices, creates tasks for message removal, session/token review, endpoint triage, and hunting for related activity. Human approval gates cover broad purge, credential reset, endpoint isolation, and blocking. Close the case only after documenting scope, uncertainty, recovery, and detection improvements.

16. Hunting, watchlists, and threat intelligence

Use a hypothesis tied to a known behavior or gap. Query relevant tables in Defender Advanced hunting and Sentinel Logs, bookmark notable Sentinel hunting results where the current experience supports bookmarks, and attach evidence to the incident. The supplied tutorial also demonstrates hunting an account manipulation technique, watchlists for enrichment, threat-intelligence indicators, and incident handoff. A watchlist can enrich a query with approved asset/IP context; do not use a broad allowlist as a substitute for evidence.

Create a watchlist from an owned CSV with clear key column, source, expiration, and review owner. Use _GetWatchlist in KQL only after verifying the watchlist alias and search key. For threat intelligence, choose an approved feed or TAXII connector, verify indicator provenance and age, search internal sightings, then decide whether a rule, hunt, or block is warranted. Shared cloud infrastructure needs narrow scoping.

17. End-to-end validation matrix

Device: onboarded, healthy sensor, recent timeline event, approved detection test. Windows: AMA healthy, DCR associated, SecurityEvent sample. Sysmon: local event, DCR filter, WindowsEvent sample, expected fields. Network: raw event agrees with parsed action/direction. Defender XDR: alert and incident, device/user entities. Sentinel: chosen table, query result, analytic alert, incident grouping. Fusion: rule enabled and source signals available; do not demand a Fusion incident from one alert. Automation: matching rule, successful Logic App run, comment, tasks, notification, errors and approvals. Operational: cost, latency, false positives, owners, shift handoff, rollback.

18. Troubleshooting and handoff

No device in inventory: inspect onboarding assignment, sensor health, connectivity, and last seen. Device present but no detection: inspect test validity, timeline, EDR policy, and alert settings. No Sentinel data: inspect source generation, AMA, DCR association, connector, table, and ingestion delay. Wrong Sysmon fields: inspect raw XML, EventData, XPath, and table schema. No correlation: verify primary workspace, connector mode, alert entities, and incident timing. No Fusion: verify eligible source signals and multistage conditions. Playbook absent: check automation rule scope/order, trigger type, permissions, Logic App connection, and run history. Unexpected action: suspend automation, preserve run history, correct mappings/approvals, and retest.

Handoff record: incident ID, current severity, affected entities, confirmed evidence, competing hypotheses, queries and time range, actions taken, pending approvals, business owner, next action, and escalation deadline.

Current Microsoft references

Sentinel onboarding https://learn.microsoft.com/en-us/azure/sentinel/quickstart-onboard

Defender portal connection https://learn.microsoft.com/en-us/azure/sentinel/microsoft-sentinel-onboard

Defender XDR integration https://learn.microsoft.com/en-us/azure/sentinel/microsoft-365-defender-sentinel-integration

Device onboarding with Intune https://learn.microsoft.com/en-us/intune/device-security/microsoft-defender/configure-integration

Windows events with AMA https://learn.microsoft.com/en-us/azure/sentinel/connect-services-windows-based

Defender process schema https://learn.microsoft.com/en-us/defender-xdr/advanced-hunting-deviceprocessevents-table

Fusion https://learn.microsoft.com/en-us/azure/sentinel/configure-fusion-rules

Automation rules https://learn.microsoft.com/en-us/azure/sentinel/create-manage-use-automation-rules

Playbooks https://learn.microsoft.com/en-us/azure/sentinel/automation/run-playbooks

Hunting https://learn.microsoft.com/en-us/azure/sentinel/hunting

Watchlists https://learn.microsoft.com/en-us/azure/sentinel/watchlists

Supplied learning material

The supplied Sentinel beginner video and five additional video transcript excerpts cover workspace setup, training lab, playbooks, data connectors, analytics, Fusion, incident management, hunting, watchlists, Defender for Endpoint onboarding, and response. This manual uses them as learning context and current Microsoft documentation for configuration details.